

## Aula 01 - O que é Versionamento e Primeiros Passos com Git

**Descrição:** Introdução ao conceito e ao histórico das metodologias de versionamento, focando em como elas evitam a perda de código em projetos de TI. A aula será totalmente prática, voltada para a instalação e configuração inicial do Git nos computadores do laboratório.

### 1. Conceito e Evolução Histórica das Metodologias de Versionamento

#### 1.1 O que é Versionamento de Código (VCS / SCM)?

- O versionamento de código (VCS – *Version Control System* ou SCM – *Source Code Management*) é definido como o ato de manter um histórico organizado de todas as versões, atualizações e modificações de um software ao longo de sua existência.
- Ele é amplamente utilizado no desenvolvimento de software para controlar o histórico de alterações tanto do código-fonte quanto da documentação.
- Trata-se de uma ferramenta indispensável para o mercado de trabalho, sendo frequentemente exigida por empresas em processos seletivos e vagas de desenvolvimento.

#### 1.2 A Analogia da "Save de Videogame"

Uma excelente alternativa para explicar o versionamento de código, que foca mais na segurança e experimentação do que no aspecto de "auditoria" é a **Analogia do Checkpoints**.

No desenvolvimento de software, escrever código é como avançar nas fases de um jogo difícil e complexo. Se você tentar jogar o jogo inteiro de uma vez só sem salvar, um único erro (ou chefe) pode fazer você perder todo o progresso e obrigá-lo a começar do zero. O versionamento de código funciona como o sistema de saves desse jogo.

Veja como os conceitos se encaixam nessa analogia:

- **Commits = Checkpoints (Pontos de Salvamento):** Cada vez que você termina uma parte importante do código (cria uma tela nova, arruma um bug), você faz um "Save" (que no Git chamamos de *Commit*). Se na semana que vem você tentar adicionar algo novo e acabar quebrando o projeto inteiro ("Game Over"), você não precisa se desesperar. Basta dar um "Load" e voltar o código exatamente para o último checkpoint onde tudo estava funcionando perfeitamente.
- **Mensagens de Commit = Nomes dos Saves:** Assim como em alguns jogos antigos você nomeava seu save para lembrar onde estava (ex: "*Antes do chefe de fogo*", "*Logo após pegar a espada*"), no versionamento você escreve uma mensagem em cada commit (ex: "*Adicionado botão de login*", "*Corrigido erro na cor do menu*"). O histórico do projeto vira a sua tela de "Carregar Jogo", onde você vê todas as etapas da sua jornada.
- **Branches = Slots de Save Alternativos (Caminhos Paralelos):** Imagine que você está jogando e chega em uma bifurcação: você quer testar um caminho perigoso, mas tem medo de estragar o seu save com a "partida perfeita". O que você faz? Copia o jogo para o "Slot 2" e testa lá.
- No Git, isso se chama criar uma *Branch* (ramificação). Você cria um caminho paralelo seguro para testar ideias novas. Se a ideia for incrível, você pode mesclar esse progresso com o seu jogo principal (processo chamado de *Merge*). Se der tudo errado, você simplesmente apaga o "Slot 2" e volta para o seu jogo principal intacto.
- **GitHub / Servidor Remoto = Multiplayer (Nuvem):** Se o jogo for cooperativo, o servidor (como o GitHub ou GitLab) é o "Save na Nuvem" que junta o progresso de todos os jogadores. Ele organiza as coisas para que o avanço do Jogador 1 não sobrescreva ou apague os itens que o Jogador 2 conseguiu, fundindo as conquistas de todos num jogo só.

#### 1.3 A Evolução Histórica das Metodologias

##### 1. Sistemas Locais de Controle de Versão (VCS Local):

- **Funcionamento:** O método tradicional consistia em copiar arquivos manualmente para outros diretórios ou manter um banco de dados simples na própria máquina local.
- **Problema:** Bastante propenso a erros operacionais e com alto risco de perda total dos dados caso o disco rígido da máquina fosse corrompido.

##### 2. Sistemas Centralizados de Controle de Versão (CVCS):

- **Exemplos:** CVS e Subversion (SVN).

- **Funcionamento:** Possuem um único servidor central que armazena todos os arquivos versionados, enquanto os clientes baixam (*checkout*) os arquivos para suas máquinas.
  - **Vantagens e Limitações:** Permitem saber o que cada pessoa da equipe está fazendo e dão controle administrativo sobre as permissões. No entanto, se o servidor central ficar fora do ar ou sofrer uma falha no disco sem backups atualizados, todo o histórico do projeto é perdido.
3. **Sistemas Distribuídos de Controle de Versão (DVCS):**
- **Exemplos:** Git, Mercurial e Bazaar.
  - **Funcionamento:** Os clientes não apenas baixam a última versão dos arquivos, mas espelham (*clonam*) o repositório completo com todo o seu histórico.
  - **Vantagem:** Cada cópia local atua como um backup completo do servidor central.

## 1.4 A Origem do Git e Linus Torvalds

A origem do Git está diretamente ligada à necessidade de gerir um dos maiores projetos de software de código aberto do mundo: o kernel do sistema operacional Linux.

### Uma Breve História do Git

Como muitas coisas na vida, o Git começou com um pouco de destruição criativa e uma controvérsia de fogo. O núcleo do Linux é um projeto de código aberto com um escopo bastante grande. A maior parte da vida da manutenção do núcleo Linux (1991-2002), as mudanças no código eram compartilhadas como correções e arquivos. Em 2002, o projeto do núcleo do Linux começou por usar uma DVCS proprietária chamada de BitKeeper.

Em 2005, a relação entre a comunidade que desenvolveu o núcleo do Linux e a empresa que desenvolveu BitKeeper quebrou, e a ferramenta passou a ser paga. Isto alertou a comunidade que desenvolvia o Linux (e especialmente Linus Torvalds, o criador do Linux) a desenvolver a sua própria ferramenta baseada em lições aprendidas ao usar o BitKeeper. Algumas das metas do novo sistema eram as seguintes:

- Velocidade
- Projeto simples
- Forte suporte para desenvolvimento não-linear (milhares de ramos paralelos)
- Completamente distribuído
- Capaz de lidar com projetos grandes como o núcleo do Linux com eficiência

Desde o seu nascimento em 2005, Git evoluiu e amadureceu para ser fácil de usar e ainda reter estas qualidades iniciais. Ele é incrivelmente rápido, é muito eficiente com projetos grandes, e ele tem um incrível sistema de ramos para desenvolvimento não linear

### Objetivos e Princípios de Design do Git

Ao criar o Git, Torvalds estabeleceu pilares fundamentais para resolver os problemas enfrentados na altura:

1. **Velocidade e Desempenho:**
  - O sistema precisava de ser extremamente rápido a executar operações e a gerir grandes projetos de software.
2. **Arquitetura de Desenvolvimento Distribuído:**
  - Em vez de depender de um servidor centralizado onde todos os desenvolvimentos tinham de ser submetidos diretamente, o Git foi projetado para ser **distribuído**.
  - Cada programador possui uma cópia integral do repositório na sua máquina local, permitindo trabalhar de forma autónoma.
3. **Ramificações Paralelas e Mesclagem (*Branches e Merges*):**
  - O Git foi desenhado para suportar milhares de ramificações paralelas (*branches*) simultâneas.
  - Isto permite que diferentes programadores trabalhem em áreas distintas do código (por exemplo, na criação de *drivers*, funcionalidades ou correção de erros) sem interferir no trabalho uns dos outros.
  - Quando o trabalho numa ramificação está concluído e testado, as alterações são integradas (*merge*) na ramificação principal.
4. **Histórico Detalhado e Rastreabilidade:**
  - O Git funciona como um registo minucioso de atualizações, guardando o histórico completo de modificações do software.
  - Em cada alteração salva (*commit*), fica registrado quem alterou, em que data a alteração foi feita, quais

os arquivos modificados e o que foi exatamente alterado em cada um.

## Filosofia *Open Source*

O Git foi disponibilizado como software **Open Source** (código aberto) e gratuito. Qualquer pessoa ou empresa pode aceder ao seu código-fonte, utilizar a ferramenta gratuitamente e contribuir para a sua evolução contínua.

## 2. Como o Versionamento Evita a Perda de Código em Projetos de TI

### 2.1 Principais Mecanismos de Proteção contra Perda de Código

- **Histórico Abrangente e Rastreabilidade:** Permite rastrear exatamente quem introduziu determinado bug ("kakinho") no sistema e quando essa alteração ocorreu, facilitando a identificação e correção de falhas.
- **Resgate de Versões Antigas e Estáveis:** Se uma nova alteração quebrar o sistema, é possível reverter o código para uma versão anterior que sabidamente funcionava.
- **Redundância e Distribuição:** Como cada desenvolvedor possui uma cópia integral do repositório em seu computador, os dados estão geograficamente distribuídos e seguros.
- **Proteção em Repositórios Remotos (Nuvem):** O envio do código para plataformas remotas, como GitHub ou GitLab, protege o projeto contra acidentes físicos na máquina local (quebra de hardware, roubos ou incêndios).
- **Reversão Simples de Erros Operacionais:** Se um arquivo ou pasta for deletado acidentalmente na máquina local, o repositório permite restaurar o estado original com comandos simples.

### 2.2 Comparação Prática: Sem Versionamento vs. Com Versionamento

- **Sem Versionamento:** O desenvolvedor faz cópias manuais de pastas com nomes como projeto\_v1, projeto\_final, projeto\_final\_mesmo. Não há registro lógico do que mudou entre uma pasta e outra, gerando confusão e perda irreparável de trechos de código descartados por engano.
- **Com Versionamento:** O projeto fica em uma única pasta limpa. O histórico de todas as alterações fica armazenado de forma transparente e organizada dentro da estrutura interna do versionador.

## 3. Arquitetura do Git, Instalação e Configuração Inicial

### 3.1 Funcionamento Técnico Interno: Diffs vs. Fotografias

- Diferente de outros sistemas antigos que salvavam cópias completas dos arquivos a cada modificação, o Git trabalha armazenando a *diff* (diferença).
- A *diff* guarda apenas as linhas que foram adicionadas, alteradas ou removidas.
- Como o Git armazena esse histórico incremental a partir do primeiro *commit* (a primeira gravação), ele consegue reconstruir o estado de qualquer arquivo em qualquer ponto do tempo.

### 3.2 Trabalho com Ramificações (*Branches*) Paralelas

- **Ramificação Principal (main ou master):** Contém a versão estável do software, pronta para uso pelo cliente final.
- **Ramificações Secundárias (Feature Branches):** Permitem que múltiplos desenvolvedores criem ramificações paralelas para trabalhar em funcionalidades específicas sem interferir no código principal.
- **Mesclagem (Merge):** Processo de juntar as alterações finalizadas e testadas de uma ramificação paralela de volta à ramificação principal.

### 3.3 Instalação

1. **Windows:** Executar o instalador do *Git* para disponibilizar o terminal **Git Bash**.
2. **Linux (Ubuntu/Debian):** Abrir o terminal e executar o gerenciador de pacotes:  
sudo apt update  
sudo apt install git

### 3. Verificação de Instalação:

No terminal/prompt, digite o comando abaixo para confirmar que o Git foi instalado corretamente:  
`git --version`

### 3.4 Configuração Inicial Obrigatória do Usuário

Antes de realizar qualquer commit, é fundamental identificar a autoria das alterações para que o histórico registre corretamente quem é o autor.

Execute os comandos a seguir no terminal do laboratório:

#### 1. Configurar o nome do desenvolvedor:

`git config --global user.name "Seu Nome Completo"`

#### 2. Configurar o e-mail do desenvolvedor:

`git config --global user.email "seu.email@exemplo.com"`

#### 3. Verificar as configurações aplicadas:

`git config --list`

## 4. Prática de Laboratório - O Ciclo Básico de Trabalho no Git

### 4.1 Entendendo as Três Áreas do Git

Compreender as três áreas principais do Git é fundamental para dominar o fluxo de trabalho desta ferramenta de controle de versões. O Git opera movendo os arquivos do seu projeto entre estes três "ambientes" locais à medida que consolida as suas alterações. Abaixo, detalhamos o que cada uma destas áreas representa e como interagem entre si:

#### 1. Working Directory (Diretório de Trabalho)

O Diretório de Trabalho é, de forma simples, a pasta física no seu computador onde os arquivos do projeto residem e estão a ser editados.

- **O que acontece aqui:** É a sua área de rascunho ou "mesa de trabalho". Quando abre os arquivos no seu editor de código e escreve novas linhas de código, elimina arquivos ou guarda modificações, todas essas ações ocorrem no Working Directory.
- **Estado dos arquivos:** O Git está sempre atento a esta pasta. Os arquivos aqui podem estar *Untracked* (arquivos novos que o Git ainda não conhece) ou *Modified* (arquivos que o Git já conhece, mas que sofreram alterações recentes).
- **Comando principal:** Para que o Git comece a preparar estas alterações para serem guardadas, utiliza-se o comando `git add <nome-do-arquivo>`.

#### 2. Staging Area / Stage (Área de Preparação)

Também conhecida tecnicamente como "Index", a Staging Area é um ambiente intermediário (virtual) entre a sua área de rascunho e a base de dados final.

- **O que acontece aqui:** Funciona como uma "zona de carga" ou um "carrinho de compras". Em vez de o Git gravar obrigatoriamente todos os arquivos que alterou no Diretório de Trabalho de uma só vez, a Staging Area permite-lhe selecionar a dedo exatamente quais os arquivos (ou até pedaços de código) que farão parte da próxima gravação oficial.
- **Vantagem:** Permite agrupar alterações de forma lógica. Por exemplo, se alterou o design da página e corrigiu um bug em arquivos completamente diferentes, pode colocar apenas os arquivos do design na Staging Area, fazer uma gravação, e depois fazer o mesmo para o bug, mantendo o histórico de trabalho limpo e compreensível.
- **Comando principal:** Para consolidar o que está na Área de Preparação, utiliza-se o comando `git commit`.

#### 3. Local Repository (Repositório Local)

O Repositório Local é a base de dados oficial do Git na sua máquina. É uma estrutura oculta (armazenada dentro da pasta `.git` na raiz do seu projeto) que contém todo o histórico de versões.

- **O que acontece aqui:** Quando executa um commit, o Git tira uma espécie de "fotografia" (*snapshot*) exata ao estado atual dos arquivos que estavam na Staging Area e guarda-os permanentemente neste repositório.
- **Segurança e Histórico:** A partir do momento em que um arquivo entra no Repositório Local através de um commit, ele está seguro. É a partir desta área que pode viajar no tempo para versões mais antigas do código, verificar quem fez o quê, criar novas ramificações (*branches*) ou desfazer erros crônicos.
- **Comando principal:** Com as alterações seguras no Repositório Local, utiliza-se o comando `git push` para sincronizar e enviar estas "fotografias" para um servidor remoto (como o GitHub ou o GitLab), compartilhando-as com a restante equipa.

#### Resumo do Fluxo de Trabalho (Workflow):

1. **Modifica** os arquivos à vontade no **Working Directory**.
2. **Prepara** (faz o *stage*) das alterações que deseja consolidar, movendo-as para a **Staging Area**.
3. **Grava** (faz o *commit*) dessas alterações selecionadas, armazenando a versão permanentemente no seu **Local Repository**.



## 4.2 Passo a Passo Prático no Terminal

**Passo 1:** Criar a pasta do projeto e inicializar o repositório

No terminal do laboratório, execute:

```
# Criar uma nova pasta para a aula
mkdir projeto-aula01
```

```
# Entrar na pasta criada
cd projeto-aula01
```

```
# Inicializar o repositório Git local
git init
```

- **O que aconteceu?** O comando `git init` cria uma pasta oculta chamada `.git`, transformando aquele diretório em um repositório controlado pelo Git.

**Passo 2:** Criar um arquivo e checar o estado (`git status`)

Crie um arquivo chamado `index.html` ou `notificacoes.txt` dentro do diretório do projeto e digite o comando:

```
git status
```

- **Explicação:** O comando `git status` exibe a situação atual do repositório. Ele mostrará que o arquivo recém-criado está no estado *Untracked* (não rastreado).

**Passo 3:** Adicionar o arquivo à área de preparação (`git add`)

Para colocar o arquivo na área de preparação (*Staging Area*):

```
# Adicionar um arquivo específico:  
git add index.html
```

```
# Ou adicionar todos os arquivos modificados da pasta:  
git add .
```

- **Explicação:** O comando `git add` seleciona os arquivos que farão parte da próxima versão registrada.

**Passo 4:** Efetivar a versão no histórico (`git commit`)

Para salvar permanentemente a versão no histórico local:

```
git commit -m "Estrutura inicial do projeto"
```

- **Explicação:** O comando `git commit` grava a "fotografia" do estado dos arquivos selecionados. O parâmetro `-m` especifica a mensagem explicativa associada àquele salvamento.

**Passo 5:** Visualizar o histórico de versões (`git log`)

Para consultar todo o histórico salvo no projeto:

```
git log
```

- **Explicação:** O comando `git log` exibe a lista de todas as versões salvas, contendo o código identificador (hash), autor, data e a mensagem descritiva de cada commit.

## **5. Exercício Prático Guiado, Boas Práticas e Encerramento**

### **5.1 Exercício Guiado de Fixação**

#### **Instruções:**

1. Abra o arquivo criado (`index.html`) e adicione duas novas linhas de código ou texto.
2. Execute `git status` no terminal e observe que o Git identificou a modificação no arquivo (*Modified*).
3. Adicione a alteração ao Stage usando `git add ..`
4. Faça o commit utilizando uma mensagem clara, por exemplo: `git commit -m "Adiciona o menu principal"`.
5. Execute `git log` e confirme que agora existem 2 commits registrados no histórico.

### **5.2 Boas Práticas para Escrita de Mensagens de Commit**

- **Seja claro e conciso:** A mensagem do histórico deve explicar de forma conceitual o que foi implementado ou alterado, e não apenas listar linhas de código.
- **Exemplo Correto:** "Adiciona verificação de saldo na conta para impedir saques sem limite".
- **Exemplo Incorreto:** "Alterado if (!haSaldo) na linha 15".

### **5.3 Resumo dos Comandos da Aula**

- `git config --global user.name / user.email:` Configura a identidade do usuário.
- `git init:` Inicia o versionamento na pasta do projeto.
- `git status:` Mostra o estado atual dos arquivos.
- `git add .:` Envia as alterações para a área de preparação (*stage*).
- `git commit -m "mensagem":` Grava permanentemente a versão no histórico.
- `git log:` Exibe o histórico de alterações gravadas.